

AGENTIC AI & LLMOPS

CS 4603 · PROGRAMMING ASSIGNMENT 2

LUMS SBASSE · SUMMER 2026

Assignment Overview

This assignment takes you through the full lifecycle of a production RAG system. You first see an end-to-end RAG pipeline running on Databricks: parsing PDFs, chunking with `ai_prep_search`, indexing into Vector Search, and building a chain with `ChatDatabricks`. Once you understand the flow, you implement it from scratch locally: building the ingestion pipeline, wiring the LCEL chain, and improving retrieval with multi-query expansion.

The core engineering decisions you will explore:

1. **Ingestion design:** how you parse, split, and embed documents determines retrieval quality downstream.
2. **Chain architecture:** the LCEL pipeline pattern (retriever, prompt, LLM, parser) is the backbone of production RAG.
3. **Retrieval optimization:** query expansion addresses vocabulary mismatch between queries and documents.

The assignment is organized around **observable outcomes**, not just implementation:

- You build working code and keep it modular.
- You inspect table schemas, chunk distributions, and retrieval scores.
- You explain what each experiment shows, not just whether it runs.
- You treat retrieval failures as part of the engineering problem, not as exceptions to ignore.

In notebooks, answer questions in markdown cells. Exploratory questions are not graded. Show all outputs: you will be graded on them.

Prerequisites

You should already be comfortable with the course material:

- Databricks and MLflow environment setup
- MLflow experiments, runs, logging, tracing, and spans
- Databricks Model Serving endpoints through the OpenAI SDK
- LLM API fundamentals, including chat completions, token streaming, and function calling

Course repository: <https://github.com/alikhawaja/cs4603>

Use the databricks tutorial held on Sunday as a reference for how LangChain integrates with Databricks.

If any of the prerequisites are unfamiliar, review the course repository and the official Databricks and MLflow documentation before starting the implementation.

Parts

- **Part 0 (05%)** is a guided setup and intro to podman. Follow it to set up your environment for the local RAG pipeline.

- **Part 1** (30%) is a guided Databricks walkthrough: parse PDFs, chunk with `ai_prep_search`, create a Vector Search index, build a RAG chain with `ChatDatabricks`. You observe, inspect, and fill in gaps.
- **Part 2** (30%) builds the local ingestion pipeline from scratch: loader, two text splitters (recursive + semantic), and a baseline pgvector collection with idempotent re-ingestion.
- **Part 3** (35%) wires the LCEL RAG chain end-to-end locally, then improves retrieval with multi-query expansion.
- **Part 4 [BONUS]** (10%) implements logical and semantic query routing.

PART

0

Environment & Infrastructure Setup [5 Points]

Overview

pgvector is a PostgreSQL extension that adds a `vector(N)` type for storing and searching embeddings. You'll use it as the local vector store for the rest of the PA after you're exposed to the core Databricks pipeline in Part 1. This Part stands up the container and gets you familiar with the vector type via hands-on SQL.

Task 0.1 Containerized PostgreSQL with pgvector [2 pts]

Get a running pgvector container on `localhost:5432` and confirm the connection works.

IMPLEMENTATION

- Follow `code-skeleton/setup.md` for the full walkthrough (Podman install, image pull, container run, Python venv)
- Confirm the container is up and accepting connections

Task 0.2 Schema inspection and vector type exploration [3 pts]

Get familiar with the `vector(N)` type before Part 1 abstracts it away. Open a psql shell and test these 4 commands interactively:

IMPLEMENTATION

- `CREATE TABLE documents (id serial PRIMARY KEY, label text, emb vector(3));`
- Three `INSERT INTO documents (label, emb) VALUES (...);` statements as a single transaction (use hand-picked 3-D points, e.g., unit vectors along the axes plus one off-axis point)
- `\d documents`
- `SELECT label, emb <=> '[0.1, 0.2, 0.3]':vector AS distance FROM documents ORDER BY distance LIMIT 1;`

The demo uses `vector(3)` for hands-on exploration. Part 1's production index uses `vector(1024)` (the dimension of `databricks-gte-large-en` embeddings).

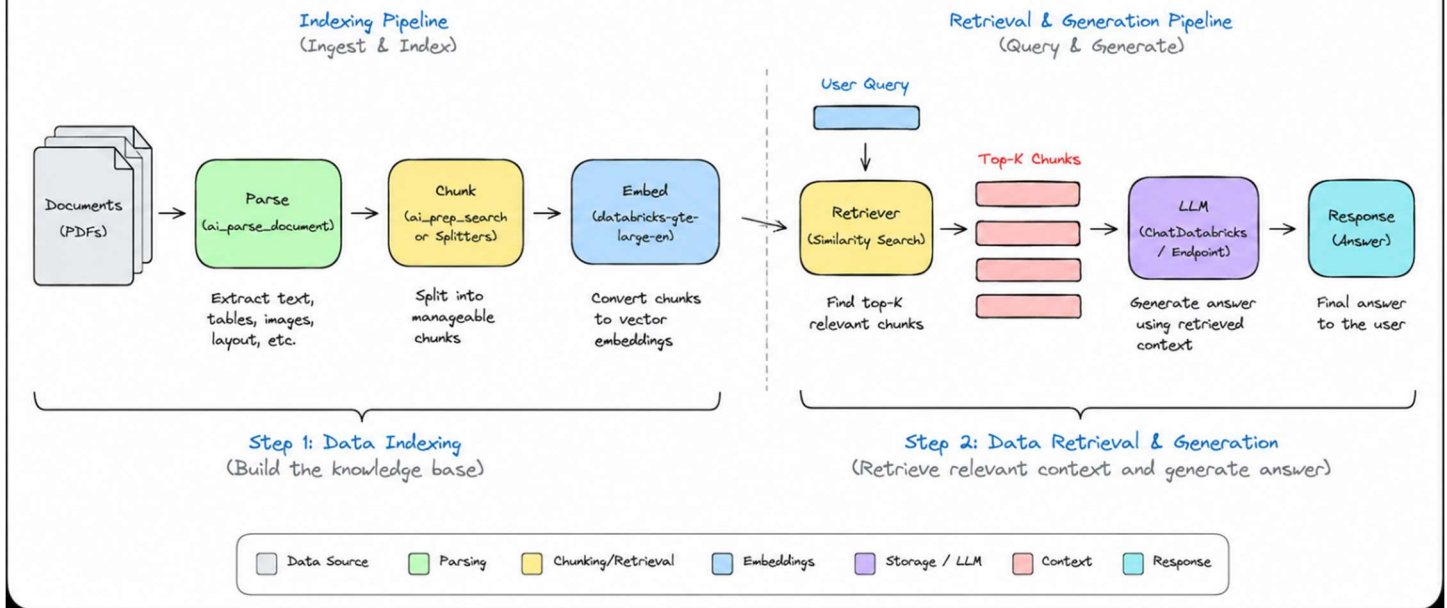
Once you've verified the commands work, create `part0.yaml` containing all 4 commands as a single YAML file using the format from the `pgvector_setup` walkthrough. After running the YAML, drop the `documents` table before moving to Part 1.

Part 0 Deliverables

File	Description
<code>part0.yaml</code>	SQL commands for pgvector schema exploration

End-to-End RAG Pipeline

Part 1



PART

1

End-to-End Databricks RAG Pipeline [30 Points]

Note: run on Databricks Platform

Notebook: `code-skeleton/notebooks/part1_databricks_rag.ipynb`

Overview

This Part walks you through a complete RAG pipeline running on Databricks. You will parse PDFs with `ai_parse_document`, chunk the parsed elements with `ai_prep_search`, create a Vector Search index with managed embeddings, retrieve relevant chunks using three different search methods, and build a full LCEL RAG chain with ChatDatabricks.

The notebook is pre-structured. Your job is to fill in the marked cells, run each cell, and observe the outputs. You will answer graded questions in markdown cells as you go.

The flow follows the natural RAG pipeline order: ingest, parse, chunk, index, retrieve, generate. Each section builds on the previous one. By the end, you will have seen the exact pattern you will implement locally in Parts 2 and 3.

Task 1.1 Environment Setup [0 pts]

Objective:

Set up Unity Catalog infrastructure and upload the 3 RAG research papers to a Volume.

REQUIREMENTS

- Create a catalog named `cs4603`
- Create a schema named `rag_data` in the catalog
- Create a volume named `corpus` in the schema
- Upload the 3 RAG research papers to the volume
- Verify the setup: run `SHOW CATALOGS`, `SHOW SCHEMAS`, `SHOW VOLUMES`, and `DESCRIBE CATALOG`

Deliverables:

- Populated volume with 3 PDF files
- Verification outputs visible in notebook

Task 1.2 PDF Parsing with `ai_parse_document` [6 pts]

Objective:

Parse PDFs into structured elements using Databricks' `ai_parse_document` function.

`ai_parse_document` takes a binary file (PDF, DOCX, image) and returns a **VARIANT**: a structured JSON object containing `document.elements` (every text block, table, figure, heading, and caption, each with a `type`, `content`, and `confidence score`) and `document.metadata` (page count, language, etc.).

REQUIREMENTS

- Create the `PARSE_TABLE` with `path STRING` and `parsed VARIANT` columns — enable Change Data Feed
- Use `ai_parse_document(content)` via `READ_FILES('/Volumes/.../corpus/', format => 'binaryFile')` to parse PDFs
- Store results in the parse table
- Explore the `VARIANT` output: use `to_json(parsed)` to convert `VARIANT` to readable string, observe element types
- Flatten elements using `LATERAL variant_explode(parsed:document.elements)` to unwrap the array into individual rows
- Display element type breakdown per PDF: which PDF has the most tables? The most text elements?

Deliverables:

- Populated parse table with `VARIANT` output
- Element type breakdown output visible in notebook

Task 1.3 Chunking with `ai_prep_search` [6 pts]

Objective:

Chunk parsed elements into retrieval-ready units with Databricks' `ai_prep_search`.

`ai_prep_search` takes the `VARIANT` output from `ai_parse_document` and produces **semantic chunks**. Semantic chunking builds upon sentence-based chunking by incorporating *semantic* relevance — ensuring the document is split based on *meaning* and *structure* rather than just character count. Each chunk includes `chunk_to_retrieve` (raw text for display) and `chunk_to_embed` (context-enriched text for embedding).

REQUIREMENTS

- Create the `CHUNK_TABLE` with columns: `chunk_id STRING`, `chunk_to_retrieve STRING`, `chunk_to_embed STRING`, `source_uri STRING` — enable Change Data Feed
- Use `ai_prep_search(parsed)` to generate chunks from the parse table
- Display chunk counts per document
- Compare `chunk_to_retrieve (raw)` vs `chunk_to_embed (enriched)` for 3 chunks
- Show chunk size distribution with a histogram

ANSWER IN MARKDOWN

- Q1: Why would the enriched version produce better search results?
- Q2: Why does `chunk_to_embed` contain more text than `chunk_to_retrieve`?

Deliverables:

- Populated chunks table
- Chunk count per document output
- Chunk size distribution histogram

Task 1.4 Vector Search Index [6 pts]

Objective:

Create a vector search index over the chunks table with managed embeddings.

A vector search index stores embeddings (numeric vectors) for each row in a Delta table and enables fast approximate nearest neighbor (ANN) search at query time. A **Delta Sync index** keeps itself in sync with the source Delta table: when new rows are inserted or updated, Databricks recomputes embeddings and updates the index automatically.

REQUIREMENTS

- Create a STANDARD vector search endpoint using `AISearchClient`
- Create a Delta Sync index with `pipeline_type="TRIGGERED", primary_key="chunk_id", embedding_source_column="chunk_to_retrieve", and embedding_model_endpoint_name="databricks-gwen3-embedding-0-6b"`
- Describe the index: show endpoint type, embedding model, status
- Verify the index is queryable with a similarity search test

ANSWER IN MARKDOWN

- **Q3:** Databricks supports two endpoint types: STANDARD and STORAGE_OPTIMIZED. What are their trade-offs?
- **Q4:** What algorithm does a STANDARD endpoint use for ANN search?
- **Q5:** What is the difference between IVFFlat and HNSW?

Deliverables:

- Created vector search endpoint
- Created Delta Sync index with status READY
- Similarity search test output

Task 1.5 Retrieval & Similarity Search [6 pts]

Objective:

Query the index using three different retrieval methods and compare the results.

Databricks supports three retrieval methods:

- **ANN** (default): semantic similarity via embedding vectors. Best for conceptual queries.
- **FULL_TEXT**: keyword matching (BM25-style). Best for exact terms, product IDs, specific identifiers.
- **HYBRID**: combines ANN and FULL_TEXT via Reciprocal Rank Fusion (RRF). Best when you need both semantic and keyword matching.

REQUIREMENTS

- Connect to the index using `DatabricksVectorSearch`
- Run 3 queries using semantic retrieval (ANN), display retrieved chunks with scores
- Run the same query using FULL_TEXT search, display results
- Run the same query using HYBRID search, display results
- Compare results across the three methods

ANSWER IN MARKDOWN

- **Q6:** In 1 line ONLY, describe what kind of query each method would be best at.
- **Q7:** Are the similarity scores across FULL_TEXT, HYBRID, and ANN comparable? Why or why not? 1 line answer ONLY.

Deliverables:

- 3 ANN retrieval outputs with scores
- FULL_TEXT retrieval output
- HYBRID retrieval output

Task 1.6 LCEL RAG Chain [6 pts]

Objective:

Build a complete RAG chain using LangChain Expression Language (LCEL).

Use LCEL to create a chain for a simple RAG pipeline. You retrieve docs, format them, insert into the prompt as context, send to llm and then parse output.

`format_docs` converts a list of LangChain `Document` objects into a single string that fits into the prompt template. It gives the RAG engineer full control over exactly what context the LLM sees: numbering, truncation, metadata stamping, filtering.

REQUIREMENTS

- Define the retriever (k=4)
- Define `format_docs(docs)` : format documents with numbered [1], [2] citations
- Define the prompt: system message that instructs the LLM to cite sources and say "I don't know" if not in context
- Build the LCEL chain
- Ask 3 questions about the RAG papers and observe the answers

ANSWER IN MARKDOWN

- **Q8:** What does `format_docs` enable a RAG engineer to do? Is the upside worth the engineering effort? Can you say the same after implementing it in Part 3?

Deliverables:

- Working RAG chain with source citations
- 3 question-answer outputs visible in notebook

Task 1.7 Understanding Embedding Space

Objective:

Visualize how chunk embeddings cluster in 2D space using UMAP.

UMAP (Uniform Manifold Approximation and Projection) reduces high-dimensional embedding vectors to 2D for visualization. UMAP captures non-linear relationships and preserves local neighborhood structure: chunks that are close in high-dimensional space stay close in 2D.

This section is ungraded. Run the cells to see the visualization and observe how chunks from the same document cluster together.

REQUIREMENTS

- Load chunks from the table
- Compute real embeddings using `DatabricksEmbeddings` pointed at the Databricks serving endpoint
- Reduce to 2D using UMAP
- Visualize with plotly scatter plot, colored by source document

Deliverables:

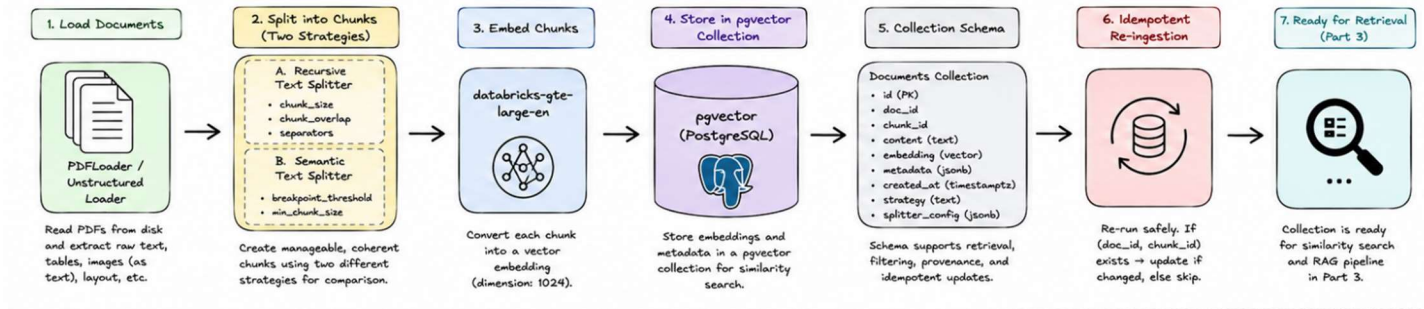
- 2D scatter plot of chunk embeddings rendered in notebook

Part 1 Deliverables

Deliverable	Description
<code>notebooks/part1_databricks_rag.ipynb</code>	Completed notebook with all outputs visible
Graded questions answered	Q1–Q8 answered in markdown cells

Part 2: Data Indexing Pipeline (Build the Knowledge Base)

Goal: Build a local ingestion pipeline that reads documents, chunks them (two strategies), embeds the chunks, and stores them in a pgvector collection with idempotent re-ingestion.



PART

2

Local Vector Storage with pgvector [30 Points]

Files: `rag/splitter.py` & `rag/store.py`

Overview

In Part 1 you saw the end-to-end Databricks RAG pipeline. Here you implement it from scratch locally: two text splitters (recursive + semantic), and a baseline pgvector collection with idempotent re-ingestion.

The core engineering decisions you will explore:

1. **Splitter choice:** recursive splitting preserves context by trying larger units first; semantic splitting groups by meaning using embedding distance.
2. **Idempotent ingestion:** the record manager ensures running the pipeline twice produces no duplicates.

Task 2.1 Recursive Text Splitter: `rag/splitter.py` [10 pts]

Objective:

Implement a recursive text splitter that tries larger units first and falls back to smaller units.

In the wk2 RAG notebook, we used LangChain's `RecursiveCharacterTextSplitter` with `chunk_size=250`, `chunk_overlap=0`. Here you reimplement that algorithm from scratch. The algorithm tries separators in order from largest to smallest (`"\n"`, `"."`, `" "`). It splits the text by the first separator. If any resulting chunk exceeds `chunk_size`, that chunk gets split again with the next separator. This continues until all chunks are under `chunk_size`. If you run out of separators, split by characters. The goal: preserve larger structural units (paragraphs, sentences) when possible, only falling back to finer splits when necessary.

IMPLEMENTATION

- `RecursiveTextSplitter` class with `chunk_size`, `chunk_overlap`, `separators` params
- `split_text(text) -> list[str]`: tries each separator in order, recurses on oversized chunks
- `split_documents(docs) -> list[Document]`: splits documents, preserves metadata from parent

Task 2.2 Semantic Splitter: `rag/splitter.py` [10 pts]

Objective:

Extend your splitter module with a semantic splitter that groups sentences by meaning.

Rule-based splitters split on syntax (paragraphs, sentences). A semantic splitter splits on *meaning*: it embeds each sentence, computes cosine distance between consecutive sentences, and breaks where the distance exceeds a threshold, indicating a topic shift.

IMPLEMENTATION

- `SemanticSplitter` class with `embeddings`, `distance_threshold`, `buffer_size` params
- `split_text(text) -> list[str]`: split into sentences, embed each, cosine distance between consecutive, group until distance exceeds threshold
- `split_documents(docs) -> list[Document]`: splits documents, preserves metadata

Hint: Split the text into sentences first (your choice of method). Buffer smooths out noise: instead of splitting whenever a single distance exceeds the threshold, you average distances over `buffer_size` consecutive pairs before deciding. For example, with `buffer_size=3`, you compute the average of distances `[d1, d2, d3]` and only split if that average exceeds the threshold.

Task 2.3 Pipeline Integration: `rag/store.py` [10 pts]

Objective:

Wire your loaders, splitters, and the record manager into a complete ingestion pipeline.

`PGVector()`, `SQLRecordManager()`, and `index()` are demonstrated in the wk2 record manager notebook. It shows all three cleanup modes (`None`, `"incremental"`, `"full"`) with hardcoded 3-document examples. Here you apply those APIs to your own loader and splitter. Use `cleanup="incremental"` only. Stale chunks for re-indexed sources are deleted, untouched sources remain.

IMPLEMENTATION

- `load_corpus(corpus_dir) -> list[Document]`: loads every PDF via `PyPDFLoader`, stamps 4-field metadata (`source`, `page`, `doc_type`, `file_hash`)
- `make_chunk_id(source, page, chunk_index) -> str`: 16-char hex SHA-256 prefix computed via `hashlib.sha256()` (capped at 16 chars)
- `build_store(documents, *, embeddings, collection_name="baseline", chunk_size=250, splitter=None)`: chunks with your splitter (defaults to `RecursiveTextSplitter` if none provided), stamps metadata["id"] + metadata["chunk_index"], indexes via the record manager with `cleanup="incremental"` and `source_id_key="file_hash"`, then ensures IVFFlat index
- `get_embeddings()` -> `DatabricksEmbeddings`: reads `EMBEDDINGS` env var, falls back to `databricks-gte-large-en`

Note: Explore the course repo and wk2 notebooks — they contain important functions such as `make_record_manager` and `make_vectorstore`, to name a few.

Key constraint: Use `cleanup="incremental"` and `source_id_key="file_hash"` only.

Downstream requirement: Every chunk **must** have `metadata["id"]` stamped by `make_chunk_id()`. Task 3.3's Reciprocal Rank Fusion deduplicates results by this field. If `metadata["id"]` is missing, RRF will raise a `ValueError`.

Part 2 Deliverables

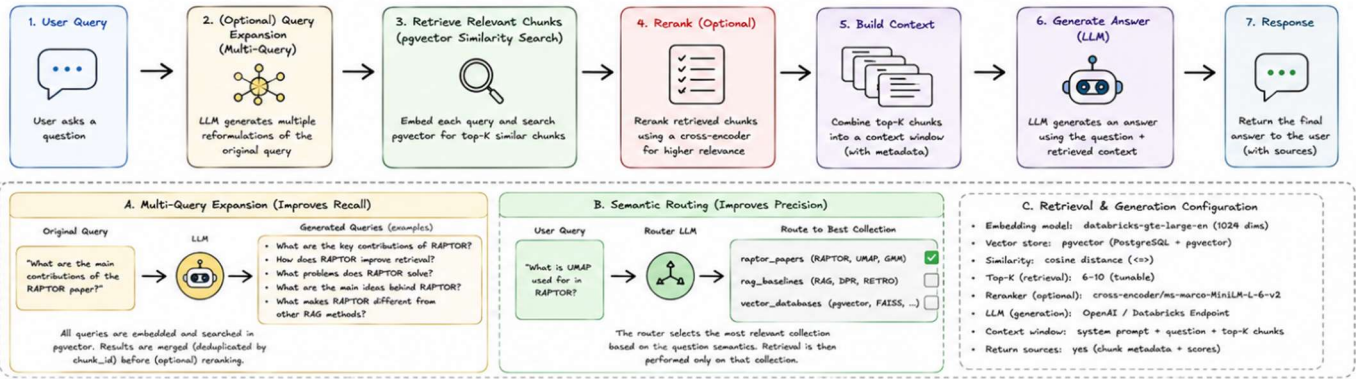
File	Description
<code>rag/splitter.py</code>	<code>RecursiveTextSplitter</code> + <code>SemanticSplitter</code>
<code>rag/store.py</code>	<code>load_corpus</code> + <code>make_chunk_id</code> + <code>build_store</code>
<code>notebooks/part2.ipynb</code>	ALL OUTPUTS SHOWN

Notebook content: Create `notebooks/part2.ipynb` (no skeleton provided). The notebook should:

1. Import and run `build_store()` to create the collection
2. Query the collection to show: table schema, chunk count per document, sample rows with `metadata["id"]` and `metadata["file_hash"]`
3. Demonstrate that re-running `build_store()` is idempotent (no duplicates)

Part 3: Retrieval & Generation Pipeline (Ask Questions, Get Answers)

Goal: Build a local RAG system that retrieves relevant context from the pgvector collection and generates accurate, grounded answers. Then improve retrieval using Multi-Query Expansion and Semantic Routing.



PART

3

RAG Chain Engineering & Adaptive Retrieval Strategies [35 Points]

Files: `rag/chain.py`, `chat/client.py`, `tools/definitions.py` & `rag/query_transform.py`

Overview

In Part 2 you built the ingestion pipeline and baseline pgvector collection. Here you wire the LCEL RAG chain against that store, expose it through two access patterns (direct mode and agentic tool), then improve retrieval with multi-query expansion.

The engineering decisions you will explore are:

- Chain architecture:** the LCEL pipeline pattern (retriever, prompt, LLM, parser) is the backbone of production RAG.
- Access patterns:** direct RAG mode vs tool call and when to use each.
- Retrieval optimization:** query expansion addresses vocabulary mismatch between queries and documents.

Important: Test your features in a notebook and show they work. Create a `notebooks/part3.ipynb` to run your code and demonstrate that each component functions correctly.

Task 3.1 LCEL RAG Chain: `rag/chain.py` [10 pts]

Objective:

Build the RAG chain using LangChain Expression Language (LCEL).

The LCEL RAG chain pattern is shown in wk2: `retrieve -> format -> prompt -> LLM -> parse`. Here you rebuild that chain with a `RunnableBranch` refusal guard, a conditional fork that short-circuits to a refusal string when retrieval returns zero docs.

Approach	Refusal reliability	LLM cost on empty retrieval
Prompt-only ("say you don't know")	~85%	Full LLM call wasted
RunnableBranch (code-level check)	100%	Zero (no LLM call)

IMPLEMENTATION

- `REFUSAL_STRING` = "I cannot answer this from the provided documents." — module-level constant
- `SYSTEM_PROMPT`: inline in `rag/chain.py` via `ChatPromptTemplate.from_messages`

- `format_docs(docs):` `numbered [N] [source: filename, page M] markers, joined "\n\n", 1-indexed pages; returns "" (no context retrieved) for empty`
- `build_rag_chain(store, *, k, chat_client, history):` LCEL pattern; `RunnableBranch` short-circuits to `REFUSAL_STRING` when retrieval is empty
- Lazy import: `from chat.client import get_chat_client` inside `build_rag_chain()` (breaks circular import)
- `get_rag_chain(k):` module-cached on `k`; `get_default_store():` module-cached
- `answer(query, *, k, history) -> {"answer": str, "docs": list[Document]}`

Key constraint: Use `ChatPromptTemplate.from_messages` with the `("placeholder", "{history}")` pattern for multi-turn support.

ANSWER IN MARKDOWN

- **Q9:** Why is branched refusal via `RunnableBranch` more reliable than prompt-only refusal? When would the ~85% prompt-only approach still be acceptable? Run your code and paste outputs as evidence.

Task 3.2 Chat Client & RAG Tool: `chat/client.py` + `tools/definitions.py` [10 pts]

Objective:

Expose the RAG chain through two access patterns: direct mode and agentic tool.

We built the chat client in PA-1 with `chat()` and `chat_loop()` methods. Here you add two ways to use RAG:

Access Pattern	Flow	Use case
<code>direct</code>	<code>chat()</code> -> LLM -> response	General questions, brainstorming
<code>rag</code>	<code>answer()</code> -> retrieve docs -> LLM -> response + sources	Document-specific questions
<code>tool</code>	LLM calls <code>rag_search()</code> during multi-turn loop	LLM decides when to retrieve

What to implement in `chat/client.py`:

IMPLEMENTATION

- `ask(question, *, mode, history):` unified entry point with `mode="direct"` or `mode="rag"`
- `mode="direct"`: delegates to `chat()` (basic chat behavior)
- `mode="rag"`: lazy-imports `answer` from `rag.chain`, retrieves + generates, prints sources
- `_print_sources(docs):` deduplicated (filename, page) citations
- Client-side refusal guard: if retrieval returns empty, print `REFUSAL_STRING` + no sources

What to implement in `tools/definitions.py`:

IMPLEMENTATION

- `rag_search(query: str) -> str:` lazy-imports `get_default_store` from `rag.chain`, builds retriever with `k=4`, invokes, joins `page_content` with `"\n\n"`
- Build schema via `build_tool_from_function(rag_search)`
- Add to `TOOLS_LIST` and `TOOL_FUNCTIONS`

Key constraint: Use lazy imports (`from rag.chain import answer` in `chat/client.py`, `from rag.chain import get_default_store` in `tools/definitions.py`) to break circular dependencies.

ANSWER IN MARKDOWN

- **Q10:** When would you use direct RAG mode vs the agentic tool pattern? What are the tradeoffs? Run both access patterns and paste outputs as evidence.

Task 3.3 Multi-Query Retrieval: `rag/query_transform.py` [15 pts]

Objective:

Expand one question into multiple variants and fuse results.

Dense retrieval fails when the query and document use different vocabulary. For example, a query about "cost reduction" might miss documents that discuss "budget optimization" or "expense management". Multi-query expansion uses an LLM to generate 3 variants of the original question, retrieves for each, and fuses via Reciprocal Rank Fusion (RRF).

Read more: Learning LangChain, Ch3. It covers query transformation techniques including Multi-Query Retrieval.

Order of operations:

```
User query
→ LLM generates 3 variant queries
→ Retrieve for each variant (batch)
→ Fuse all ranked lists via RRF
→ Dedupe by metadata["id"]
→ Return top-k merged results
```

RRF formula: $score(d) = \sum (1 / (rank + k))$ across all ranked lists, where $k=60$ (smoothing constant from the original paper — you can try and vary this in `notebooks/part3.ipynb` to achieve better results).

IMPLEMENTATION

- `QueryList` Pydantic model: structured output for query expansion — single field `queries: list[str]`
- `QUERY_EXPANSION_SYSTEM / QUERY_EXPANSION_HUMAN`: inline prompt templates — system prompt instructs LLM to generate N alternative queries; human prompt passes the question and requests JSON list output
- `parse_queries_output(message)`: extract query list from LLM response (handle both `QueryList` and raw JSON)
- `make_query_gen(llm, prompt): LCEL sub-chain: prompt -> llm.with_structured_output(QueryList) -> parse_queries_output`
- `reciprocal_rank_fusion(results, k=60)`: fuse multiple ranked document lists; **must dedupe by `metadata["id"]`**
- `MultiQueryRetriever(BaseRetriever)`: wraps base retriever with LLM query expansion + RRF fusion
- — `model_post_init` builds the query expansion chain
- — `_get_relevant_documents` implements: expand → batch retrieve → fuse

Key constraint: All documents must have `metadata["id"]` stamped by `make_chunk_id()` in Task 2.3 for RRF deduplication to work.

ANSWER IN MARKDOWN

- **Q11:** When did multi-query help, and when did it not? What types of queries benefit most? Run single-query vs multi-query on 3 questions and paste outputs as evidence.

Part 3 Deliverables

File	Description
rag/chain.py	<code>REFUSAL_STRING + SYSTEM_PROMPT + format_docs + build_rag_chain (with RunnableBranch) + get_rag_chain + get_default_store + answer</code>
chat/client.py + tools/definitions.py	<code>ask(mode="direct" "rag") + _print_sources + rag_search + rag_tool</code> in <code>TOOLS_LIST/TOOL_FUNCTIONS</code>
rag/query_transform.py	<code>MultiQueryRetriever + QueryList + reciprocal_rank_fusion</code>
notebooks/part3.ipynb	Q9–Q11 answers with code evidence + ALL OUTPUTS SHOWN

Notebook deliverable: Create `notebooks/part3.ipynb` answering Q9–Q11. For each question, run your code and paste outputs as evidence. Use markdown cells for explanations and code cells for the evidence.

File: `rag/router.py`

Objective: Route user queries to domain-specific prompts using two strategies: logical routing (LLM classification) and semantic routing (embedding similarity).

Logical routing uses an LLM to classify the query into a data source. Semantic routing embeds prompt templates and uses cosine similarity — this is faster and removes the LLM call requirement.

Read more: Learning LangChain, Ch3. It covers Query Routing strategies.

How each works:

```
Logical routing:
  User query → LLM classifies into domain → PromptTemplate → LLM → response

Semantic routing:
  User query → Embed query → Cosine similarity with prompt embeddings → PromptTemplate →
  LLM → response
```

Shared domain prompts (SEMANTIC_ROUTER_PROMPTS):

```
SEMANTIC_ROUTER_PROMPTS: dict[str, str] = {
    "physics": "You are a very smart physics professor...",
    "math":    "You are a very good mathematician...",
    "cs":      "You are a knowledgeable computer science professor...",
}
```

Task 4.1 Logical routing [5 pts]

Objective:

Use an LLM with structured output to classify queries into domains.

Define a Pydantic schema, use `llm.with_structured_output()`, then map the result to a prompt template.

IMPLEMENTATION

- `RouteQuery` Pydantic model with a single field `datasource: Literal["physics", "math", "cs"]`
- `LOGICAL_ROUTE_SYSTEM`: system prompt instructing the LLM to route by subject
- `choose_route(result: RouteQuery) -> PromptTemplate`: maps `result.datasource` to the corresponding template from `ROUTER_PROMPTS`
- `build_logical_router(llm=None): factory returning: prompt -> llm.with_structured_output(RouteQuery) -> choose_route`

Key constraint: Use `ChatPromptTemplate.from_messages` with system + human pattern. The factory defaults `llm` to `get_chat_llm()` from `utils.client`.

Task 4.2 Semantic routing [5 pts]

Objective:

Route queries using embedding similarity to remove the LLM call needed for routing.

Embed prompt templates, compute cosine similarity with the query embedding, pick the most similar prompt.

IMPLEMENTATION

- `SemanticRouter` class:
- `__init__(self, prompts, embeddings)`: embed all prompt template values via `embeddings.embed_documents()`
- `route(self, query) -> str`: return the domain key with highest cosine similarity
- `get_prompt(self, query) -> PromptTemplate`: return the `PromptTemplate` for the matched domain

- `build_semantic_router(embeddings=None, llm=None, prompts=None): factory returning: router.get_prompt -> llm -> StrOutputParser`

Key constraint: Use `cosine_similarity` from `langchain.utils.math`. Factory defaults `embeddings` to `get_embeddings()` from `rag.store` and `llm` to `get_chat_llm()` from `utils.client`.

Testing both routers — Answer in `notebooks/part4.ipynb`:

Create `notebooks/part4.ipynb` (no skeleton provided). Test both routers side by side:

1. Route 3+ queries from different domains through each router
2. Show which domain each router chose for each query
3. Show the LLM response for each
4. Compare: when do logical and semantic routing agree or disagree?

Part 4 Deliverables

File	Description
<code>rag/router.py</code>	<code>RouteQuery + choose_route + build_logical_router + SemanticRouter + build_semantic_router</code>
<code>notebooks/part4.ipynb</code>	Both routers tested side by side across domains + ALL OUTPUTS SHOWN

Grading Rubric

Component	Points
Part 0: Environment & Infrastructure Setup	5
Part 1: End-to-End Databricks RAG Pipeline	30
Part 2: Local Vector Storage with pgvector	30
Part 3: RAG Chain Engineering & Adaptive Retrieval Strategies	35
Part 4: Query Routing [BONUS]	10
Total	100

Submission

Submit a single zip file containing:

- All source code in the prescribed directory structure
- Your completed notebooks with every (graded) question answered and outputs shown
- Do **not** include `.env`, virtual environment directories, or large binary/model files

Name your submission: `<roll_number>_pa2.zip`